



Universidad
Indoamérica

UNITAT 5 · FONAMENTS DE SISTEMES OPERATIUS

Canonades i composició de processos

Universidad Tecnológica Indoamérica

Laboratori TD4

Com es comuniquen dos processos aïllats?

Fluxos de bytes

Canonades (aquesta unitat), FIFO (unitat 3), sòcols (unitat 8)

Notificacions

Senyals (unitat 6)

Memòria compartida

`mmap` i memòria compartida (unitat 6), fils (unitat 7)

La canonada és la més simple i la més usada: cada `|` del shell n'és una.

Una canonada: `pipe(fds)`

- Crea una memòria intermèdia al nucli i dos descriptors: `fds[0]` per llegir i `fds[1]` per escriure
- Una sola direcció, un flux de bytes sense missatges i sense nom
- Només la comparteixen els processos que hereten els descriptors: **es crea abans de `fork`**
- Cada procés **tanca l'extrem que no fa servir**

C tds/td4/mypipe.c

```
exit_if(pipe(fds) == -1, "pipe");
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    exit_if(close(fds[1]) == -1, "close"); /* ch.
    char buffer[READ_MAX];
    ssize_t n = read(fds[0], buffer, READ_MAX); /* ne
    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n)
    fprintf(stderr, "I am the child, I read %zd bytes an
    exit_if(close(fds[0]) == -1, "close");
    return EXIT_SUCCESS;
```

Les regles d'una canonada

Llegir amb la canonada buida	es bloqueja... o retorna 0 si no queda cap escriptor
Escriure amb la canonada plena	es bloqueja fins que algú llegeix
Escriure sense cap lector	senyal SIGPIPE (o EPIPE si s'ignora)
Escriure ≤ PIPE_BUF (4096 B)	atòmic: no es barreja amb els altres

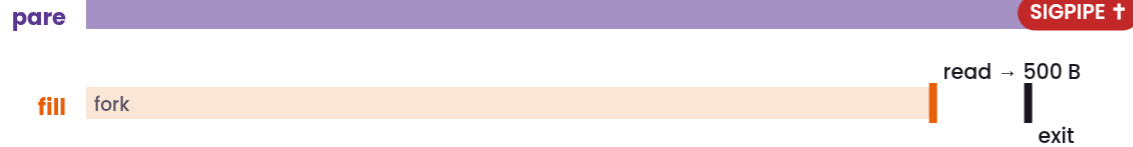
Capacitat per defecte a la màquina del curs:

Sortida real

```
$ python3 -c 'import os, fcntl; r, w = os.pipe();
print(fcntl.fcntl(w, fcntl.F_GETPIPE_SZ), "bytes")'
65536 bytes
```

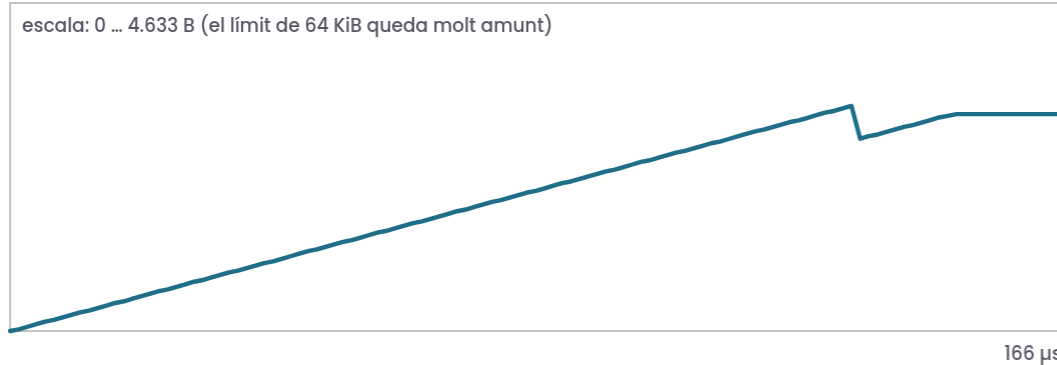
10, 300, 10.000 missatges: què passa?

línia de temps (model, µs)

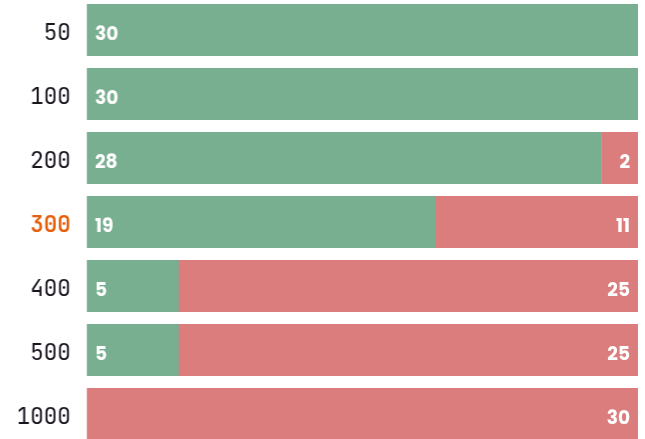


bytes a la canonada

escala: 0 ... 4.633 B (el límit de 64 KiB queda molt amunt)



mesura real: 30 execucions per mida



verd: estat 0 · vermell: 141 (SIGPIPE)
missatges

Tancar o no `fds[0]` al pare

Sortida real

```
$ ./mypipe 10000 > /dev/null; echo "exit status of mypipe: $?"  
I am the child, I read 500 bytes and I leave  
exit status of mypipe: 141
```

Sortida real

```
$ timeout 3 ./mypipe_bug 10000 > /dev/null; echo "exit status: $? (124 = killed by timeout: it was blocked)"  
I am the child, I read 500 bytes and I leave  
exit status: 124 (124 = killed by timeout: it was blocked)
```

Amb l'extrem tancat, el pare rep SIGPIPE (141 = 128 + 13) quan el fill se'n va. Sense tancar-lo, el pare és el seu propi «lector»: omple els 64 KiB i es bloqueja per sempre (124 = mort per `timeout`). En executar-lo amb `&`, el shell informa de l'estat de sortida.

Endollar el programa d'un altre: `dup2` + `exec`

C tds/td4/upper.c

```
/* parent */
exit_if(close(fds[0]) == -1, "close");
exit_if(dup2(fds[1], STDOUT_FILENO) == -1, "dup2");
exit_if(close(fds[1]) == -1, "close");          /* 1 already closed */
execlp("ps", "ps", (char *)NULL);              /* "ps" to run */
exit_if(1, "execlp ps");
```

Sortida real

```
$ ./upper | head -4
  PID TTY          TIME CMD
 6706 ?            00:00:25 UVICORN
 9771 ?            00:00:03 SYSTEMD
 9801 ?            00:00:00 (SD-PAM)
```

`ps` no sap res de canonades: escriu al seu descriptor 1, que `dup2` ha connectat a la canonada. El fill llegeix fins al final de fitxer i ho passa a majúscules.

Què fa el shell amb `ls -l | tr -d '[:blank:]'`

1. `pipe(fds)`
2. `fork` → fill 1: `dup2(fds[1], 1)`, tanca tots dos, `exec("ls")`
3. `fork` → fill 2: `dup2(fds[0], 0)`, tanca tots dos, `exec("tr")`
4. El shell tanca els seus dos extrems i espera els dos fills

Les taules de descriptors, pas a pas

pipeline

pipeline

0	terminal
1	terminal
2	terminal
3	
4	
5	
6	

fill 1

ls -l demo

0	terminal
1	canonada 0 · escriptura
2	terminal
3	
4	
5	
6	

fill 2

grep txt

0	canonada 0 · lectura
1	canonada 1 · escriptura
2	terminal
3	
4	
5	
6	

fill 3

wc -l

0	canonada 1 · lectura
1	terminal
2	terminal
3	
4	
5	
6	

canonada 0 · escriptors 1 · lectors 1

canonada 1 · escriptors 1 · lectors 1

línia 60: close(4)

N ordres i l'estat del pipeline

C tds/td4/pipeline_args.c

```
if (pid == 0) {
    if (in != STDIN_FILENO) {
        exit_if(dup2(in, STDIN_FILENO) == -1, "dup2");
        exit_if(close(in) == -1, "close");
    }
    if (fds[1] != -1) {
        exit_if(dup2(fds[1], STDOUT_FILENO) == -1,
            exit_if(close(fds[0]) == -1 || close(fds[1])
        }
        execvp(commands[i][0], commands[i]);
        perror(commands[i][0]);
        _exit(127);
    }
    if (in != STDIN_FILENO)
        exit_if(close(in) == -1, "close");
    if (fds[1] != -1) {
        exit_if(close(fds[1]) == -1, "close");
        in = fds[0];
    }
}
```

Sortida real

```
$ ./pipeline_args ls -l demo -- grep txt -- wc -l
2
```

Sortida real

```
$ ./pipeline_args ls -- sh -c 'exit 3'; echo "status: $?"
status: 3
```

Sortida real

```
$ ./pipeline_args ls -- sh -c 'kill -TERM $$'
pipeline: last command killed by SIGTERM
[exit status 1]
```

Com el shell, retorna l'estat de l'última ordre
(`set -o pipefail` canvia aquesta regla a
bash).

Més enllà de `pipe`

`pipe2`

`O_CLOEXEC` evita que la canonada s'escoli als programes llançats amb `exec` ; `O_NONBLOCK` per no bloquejar-se.

`splice`

Mou dades entre una canonada i un fitxer o sòcol sense copiar-les al procés.

Capacitat

`fcntl(F_SETPIPE_SZ)`
l'amplia fins a
`/proc/sys/fs/pipe-max-size`.

TD4 resol't + repte IA

- `mypipe` amb 10, 300 i 10.000 missatges; amb l'error i sense
- `upper` : `ps` en majúscules
- `pipeline_args` `ls -l -- grep txt -- wc -l`

Bash

```
make -C tds && tds/tests/run.sh td4
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Quan retorna 0 un `read` sobre una canonada?
2. Per què `mypipe 10000` acaba amb 141 i `mypipe_bug` no acaba mai?
3. A `a | b | c`, quantes crides a `pipe`, `fork` i `close` fa el shell?